connection authenticate each other cryptographically. Identity flows through the mesh transparently. You write your service like it's calling plain HTTP, and the mesh handles the encryption and identity verification.

If you're using Istio, Linkerd, or Cilium, you get a lot of this for free. Worth setting up if you haven't. The first time you watch a service refuse to talk to another service that isn't supposed to be calling it, you'll wonder how you ever lived without it.

The harder part again is that microservices multiply the number of places where things can go wrong. Every internal endpoint is a potential entry point if an attacker manages to land somewhere in your cluster. So the same care you'd put into public endpoints needs to extend inward. No service should trust another service just because they share a network.

Rate limiting, the boring superpower

Nothing about rate limiting is exciting. Nobody puts "implemented sophisticated rate limiting" on their resume. And yet I would bet that more breaches and abuse incidents are prevented by a thoughtful rate limit than by any other single technique.

The idea is simple. Limit how many requests a given caller can make in a given window. If they exceed it, slow them down or reject them outright. This blunts pretty much every automated attack. Credential stuffing. Scraping. Enumeration. Brute force. All of them depend on being able to make a lot of requests quickly. Take that away and the attacker gets bored and moves on.

The trick is that rate limits aren't one-size-fits-all. A user logging in might reasonably make five login attempts in a few minutes. An automation pulling reports might reasonably make a thousand requests in a few minutes. A public unauthenticated endpoint should be much more restricted than an authenticated one.

Tools like NGINX, Envoy, or cloud-managed gateways can do this for you. Or you can do it in your application using libraries like express-rate-limit for Node or django-ratelimit for Django. Wherever you put it, just put it somewhere.

A reasonable starting point might look like this in Express:

```
const rateLimit = require('express-rate-limit');

const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: 'Too many login attempts. Please try again later.'
});

app.post('/api/login', loginLimiter, handleLogin);
```

Five attempts per fifteen minutes for login. That single change makes credential stuffing impractical for most attackers.

Input validation and the lie of frontend safety

I've said this in the previous article (*OSFY*, June 2026) and I'll say it again because it bears repeating. Frontend validation is a UX feature, not a security feature. Anyone with curl, or these days anyone with an LLM and ten minutes, can skip your beautiful form and hit your API with whatever shape of data they want.

Validate everything on the server. Use a schema validator like Zod, Joi, Yup, or Pydantic for Python. Define exactly what shape each endpoint accepts. Reject anything that doesn't match. Don't try to be clever. Don't try to coerce weird input into the right shape. Just reject it.

This sounds obvious, but I see APIs that happily accept extra fields and stick them straight into a database insert without checking. That's how you get mass assignment vulnerabilities, where a user sends *{ "name": "Alice", "isAdmin": true }* and your endpoint dutifully grants them admin rights because nobody filtered the input.

The pattern is simple. Define the shape you expect. Strip everything else. Trust nothing.

```
const userUpdateSchema = z.object({
  name: z.string().min(1).max(100),
  email: z.string().email(),
  preferences: z.object({
    theme: z.enum(['light', 'dark']).optional()
  }).optional()
}).strict();
```

That *.strict()* at the end is the magic. It rejects requests with unexpected fields, instead of silently passing them through.

Logging without leaking, monitoring without panicking

You need logs to figure out what happened. But logs can become a security problem if you're not careful.

The most common mistake is logging request bodies wholesale. Some of those bodies contain passwords, API keys, credit card numbers, personal data. Once that ends up in your log aggregator, you've effectively expanded the scope of where that sensitive data lives. Now your logging system needs the same protections as your database.

A better approach is to decide ahead of time what's worth logging. Method, path, status code, response time, the identity of the caller, any error code. Skip the body unless you've explicitly scrubbed it of sensitive fields. Some logging libraries have built-in redaction. Use it.

On the monitoring side, watch for patterns rather than individual events. A single 401 doesn't mean anything. A thousand 401s from one IP in five minutes means something. A user account that suddenly starts hitting endpoints it has never hit before,